

clautolisp - Common Lisp AutoLISP Runtime & IDE - Specification Draft

Converted from PDF

June 24, 2026

Contents

1	Goal	1
2	Vision	1
3	Primary Goals	1
4	Architecture Overview	2
5	Subsystem Estimates	2
5.1	alisp-core	2
5.2	alisp-vl	2
5.3	alisp-dcl	2
5.4	alisp-host-*	3
6	Runtime Design	3
7	Testing Strategy	3
8	Out-of-Process UI Model	4
9	IDE Features	4
10	Migration Path Toward Common Lisp	4
11	Future Extensions	5

1 Goal

Offline execution of legacy AutoLISP libraries, CAD regression testing, and a modern development environment.

2 Vision

Create a portable AutoLISP runtime implemented in Common Lisp that can run legacy AutoLISP libraries offline, provide strong debugging and testing tools, and gradually allow migration toward Common Lisp.

The environment must support headless execution for CI and regression testing, and allow UI simulation without requiring a graphical CAD application.

3 Primary Goals

- Run legacy AutoLISP libraries outside AutoCAD/BricsCAD.
- Provide a modern AutoLISP IDE with debugging, inspection, and testing.
- Enable CAD regression testing through deterministic snapshots.
- Support headless execution in CI pipelines.
- Provide an evolutionary path from AutoLISP to Common Lisp.

4 Architecture Overview

The system is divided into several subsystems designed for portability and testability:

- `alisp-core`: language runtime and evaluator.
- `alisp-vl`: Visual LISP compatibility layer.
- `alisp-dcl`: dialog and UI abstraction.
- `alisp-host-*`: bridges to CAD environments or testing backends.

5 Subsystem Estimates

5.1 **alisp-core**

- Language evaluator.
- Symbol and environment model.
- Numeric, list, and string primitives.
- Error system mapped to Common Lisp conditions.
- Estimated public functions: 120-200.

5.2 **alisp-vl**

- Visual LISP helper functions such as `vl-string-*`, `vl-file-*`, and related APIs.
- Estimated public functions: 180-350.

5.3 **alisp-dcl**

- Dialog parsing.
- Tile manipulation.
- Callback wiring.
- Estimated public functions: 60-120.

5.4 **alisp-host-***

- CAD entity access.
- Selection sets.
- Snapshot and diff testing.
- Estimated public functions: 80-180.

6 Runtime Design

The AutoLISP runtime is implemented as a Common Lisp evaluator rather than by translating code into Common Lisp.

This preserves AutoLISP semantics including:

- Dynamic scoping.
- AutoLISP symbol rules.
- Built-in function behavior.
- Error handling via `*error*`.

7 Testing Strategy

Regression testing is based on entity snapshots.

Test workflow:

1. Capture a snapshot of the drawing state.
2. Execute AutoLISP code.
3. Capture the resulting snapshot.
4. Compare differences using deep, shallow, or count-based modes.

A mock host backend allows testing without CAD.

8 Out-of-Process UI Model

UI runs as a separate process using Qt6.

Advantages:

- Headless testing remains possible.
- UI crashes do not affect the runtime.
- Portability across macOS, Windows, and Linux is simpler.

Communication between runtime and UI uses a simple message protocol, for example JSON or MsgPack.

9 IDE Features

The environment integrates with SLIME or a similar REPL-based workflow.

Capabilities:

- Interactive debugging.
- Breakpoints.
- Environment inspection.
- Stepping.
- Regression tests.
- Snapshot diff tools.

10 Migration Path Toward Common Lisp

The runtime allows gradual migration:

1. Run existing AutoLISP code unchanged.
2. Introduce optional Common Lisp libraries.
3. Gradually replace AutoLISP constructs with Common Lisp equivalents.

This should eventually enable hybrid AutoLISP/Common Lisp applications.

11 Future Extensions

- CAD integration backends for AutoCAD and BricsCAD.
- DCL compatibility layer.
- Visual debugging tools.
- Graphical diff support for drawing changes.